



SUPPORT DE COURS COMPLET

Security by design

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction : principes de security by design et privacy by design
2. Chapitre 2 — Modélisation des menaces (threat modeling)
3. Chapitre 3 — Principes de conception sécurisée
4. Chapitre 4 — Architecture sécurisée : segmentation et zero trust
5. Chapitre 5 — Cycle de développement sécurisé (DevSecOps)
6. Chapitre 6 — Privacy by design

Security by design

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Comprendre les principes fondamentaux de conception sécurisée, à intégrer dès la phase de conception plutôt qu'ajoutés a posteriori.
- Maîtriser une démarche de modélisation des menaces (threat modeling).
- Comprendre les architectures de sécurité modernes (défense en profondeur, zero trust).
- Intégrer la sécurité dans le cycle de développement logiciel (DevSecOps).
- Comprendre les principes de *privacy by design*.

Prérequis

Ce module s'appuie sur les notions vues en *Audit organisation et technique* (OWASP Top 10, méthodologie d'audit) et bénéficie d'une bonne compréhension du développement logiciel (module *Algorithmique et Programmation en C*).

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction : principes de security by design et privacy by design	2h
2	Modélisation des menaces (threat modeling)	3h
3	Principes de conception sécurisée	3h
4	Architecture sécurisée : segmentation et zero trust	3h
5	Cycle de développement sécurisé (DevSecOps)	2h
6	Privacy by design	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Threat modeling STRIDE sur une application	2h
2	Application des principes de conception sécurisée	2h

Séance	Sujet	Durée
3	Conception d'une architecture zero trust	2h
4	Intégration de contrôles de sécurité en CI/CD (DevSecOps)	2h

Évaluation

- TP notés : 40 %
- Étude de cas de threat modeling notée : 20 %
- Examen final : 40 %

Bibliographie

- A. Shostack, *Threat Modeling: Designing for Security*.
- OWASP Top 10, OWASP Application Security Verification Standard (ASVS).
- NIST SP 800-207 — Zero Trust Architecture.
- CNIL / autorités de protection des données — guides *Privacy by Design*.

Chapitre 1 — Introduction : principes de security by design et privacy by design

1. Définition

Security by design désigne l'approche consistant à intégrer les exigences de sécurité **dès la phase de conception** d'un système, plutôt que de les ajouter a posteriori sous forme de correctifs après un incident ou un audit. C'est un changement de posture : la sécurité devient une contrainte de conception au même titre que la performance ou l'ergonomie, pas une couche additionnelle optionnelle.

Cette approche s'oppose à ce que l'on appelle parfois la « sécurité en pansement » (*bolt-on security*) : un système conçu sans considération de sécurité, auquel on ajoute ensuite des dispositifs de protection externes (pare-feu, antivirus, module d'authentification greffé après coup) pour compenser des faiblesses structurelles. Le problème de cette approche corrective n'est pas seulement son coût, mais son efficacité intrinsèquement limitée : un dispositif ajouté après coup ne peut compenser que les symptômes visibles d'un problème de conception, pas sa cause racine. Un système dont l'architecture logique mélange par exemple code métier et requêtes SQL construites par concaténation de chaînes reste vulnérable à l'injection quelle que soit la sophistication du pare-feu applicatif placé devant lui.

Security by design repose sur trois idées structurantes que le reste de ce module développe :

- la sécurité est une **propriété du système**, résultant de choix de conception, pas seulement de la présence de mécanismes de protection ;
- elle doit être pensée **en amont**, dès la définition des exigences et de l'architecture, et non uniquement lors des tests ou de l'exploitation ;
- elle nécessite une **démarche systématique et reproductible** (modélisation des menaces, principes de conception éprouvés, processus outillé), pas seulement la vigilance individuelle de développeurs expérimentés.

2. Pourquoi corriger après coup coûte plus cher

Le coût de correction d'une faille de conception augmente très fortement à mesure que le projet avance : une faille identifiée en phase de conception coûte typiquement un ordre de grandeur de moins à corriger qu'une faille découverte en production, en particulier si elle nécessite de repenser une architecture déjà déployée et adoptée par des utilisateurs. Cette observation motive l'intégration précoce de la sécurité (« shift left »), reprise au chapitre 5 (DevSecOps).

Plusieurs mécanismes concrets expliquent cette augmentation du coût au fil du cycle de vie :

- **Effet d'engagement architectural** : une décision de conception initiale (par exemple, l'absence de séparation claire entre les composants de confiance et les composants exposés) se propage dans de nombreux modules qui en dépendent implicitement. La corriger tardivement impose souvent de retoucher un grand nombre de composants interconnectés, alors qu'elle aurait pu être évitée par un simple choix différent au moment de la conception.

- **Effet de régression** : plus un système est déployé et utilisé, plus une modification structurelle risque de casser des comportements existants sur lesquels des utilisateurs ou des systèmes tiers se sont appuyés (compatibilité ascendante, intégrations externes), ce qui alourdit les tests de non-régression nécessaires.
- **Effet d'urgence post-incident** : une correction menée dans l'urgence après un incident de sécurité avéré se fait souvent sous pression, avec moins de recul et de tests que dans un contexte de conception planifiée, ce qui augmente le risque d'introduire de nouveaux défauts en corrigeant le premier.
- **Coût organisationnel additionnel** : un incident de sécurité découvert en production entraîne typiquement des coûts qui dépassent le seul correctif technique (communication de crise, notification éventuelle des autorités ou des personnes concernées selon la réglementation applicable, perte de confiance des utilisateurs, temps d'immobilisation d'équipes entières pour l'investigation).

Illustration pédagogique (cas fictif)

Considérons un établissement fictif, « BanqueTest SA », qui développe une application de gestion de comptes. Si l'équipe de conception choisit dès le départ de valider systématiquement l'identité du propriétaire d'un compte avant chaque opération sensible (médiation complète, voir chapitre 3), le coût de cette vérification est négligeable : quelques lignes de code additionnelles dans une couche d'accès aux données déjà centralisée. Si en revanche cette vérification est omise à la conception et qu'une faille d'accès non autorisé à des comptes tiers est découverte après la mise en production, la correction impose de retoucher potentiellement des dizaines de points d'accès aux données disséminés dans l'application, de rejouer des tests de sécurité complets, et de gérer la communication envers les clients concernés.

3. Security by design vs sécurité périmétrique

Un modèle de sécurité historique reposait sur une **frontière protégée** (pare-feu, périmètre réseau) protégeant un intérieur considéré comme de confiance. Ce modèle s'avère fragile dès qu'un attaquant franchit le périmètre (par hameçonnage, compromission d'un poste, prestataire tiers compromis) : à l'intérieur, peu de contrôles supplémentaires ralentissent sa progression. *Security by design* pousse à concevoir des systèmes résilients même en présence d'une compromission partielle, préfigurant le modèle zero trust (chapitre 4).

L'image souvent utilisée est celle du « château fort contre l'œuf » : un château fort protégé par des murailles épaisses mais dont l'intérieur est peu compartimenté s'effondre rapidement une fois la muraille franchie ; un œuf, en apparence plus fragile, oppose en réalité une résistance structurelle répartie sur toute sa coquille. *Security by design* cherche à construire des systèmes plus proches du second modèle : la résistance à la compromission n'est pas concentrée sur un unique point de contrôle périphérique, mais répartie à travers l'ensemble de l'architecture (moindre privilège, segmentation, médiation complète à chaque frontière interne, pas seulement à la frontière externe).

4. Les grands principes, aperçu

Ce module détaille au chapitre 3 une liste de principes de conception sécurisée reconnus (moindre privilège, défense en profondeur, échec sécurisé, séparation des tâches, simplicité de conception, médiation complète), formalisés notamment par Saltzer et Schroeder dès 1975 et toujours d'actualité. Ces principes ne sont pas spécifiques à une technologie ou un langage de programmation particulier : ils s'appliquent aussi bien à la conception d'un système d'exploitation, d'une application web moderne, que d'une architecture Cloud distribuée. Leur ancienneté relative (près de cinquante ans pour les travaux fondateurs) n'est pas un signe d'obsolescence mais, au contraire, un indice de leur robustesse conceptuelle : ils décrivent des propriétés structurelles des systèmes, indépendantes des modes technologiques.

5. Privacy by design

Concept complémentaire, formalisé par Ann Cavoukian (fin des années 1990-2000) et aujourd'hui explicitement exigé par plusieurs réglementations de protection des données (ex. article 25 du RGPD, « protection des données dès la conception et par défaut ») : la protection des données personnelles doit être une caractéristique par défaut d'un système, pas une option activable. Ce concept est développé au chapitre 6.

Il est utile de distinguer d'emblée *security by design* et *privacy by design* : le premier vise la protection du système et de ses actifs contre des actions malveillantes (confidentialité, intégrité, disponibilité au sens large) ; le second vise spécifiquement la protection des personnes dont les données sont traitées, y compris contre des usages parfaitement licites mais excessifs (collecte disproportionnée, conservation trop longue, profilage non désiré). Un système peut être techniquement très sécurisé (chiffrement fort, contrôle d'accès rigoureux) tout en étant peu respectueux de la vie privée s'il collecte et conserve, de façon sécurisée, bien plus de données personnelles que nécessaire. Les deux démarches sont complémentaires, pas substituables l'une à l'autre.

6. Sécurité et fonctionnalité : un arbitrage, pas une opposition

Une idée reçue oppose sécurité et facilité d'usage. Une conception mature cherche plutôt à rendre le **comportement sécurisé le comportement par défaut et le plus simple** pour l'utilisateur (ex. authentification à double facteur activée par défaut plutôt que reléguée à un réglage avancé rarement découvert). Une mesure de sécurité qui dégrade excessivement l'expérience utilisateur est souvent contournée en pratique (mots de passe notés sur un post-it, désactivation d'un contrôle jugé trop contraignant) — un échec de conception, pas seulement un problème de discipline des utilisateurs.

Ce constat rejoint un principe plus général de conception centrée sur l'humain (*usable security*) : un contrôle de sécurité qui n'est pas utilisé, ou qui est contourné, a une efficacité pratique nulle voire négative (il crée un faux sentiment de protection). Concevoir un mécanisme de sécurité implique donc de considérer, dès l'origine, le parcours réel de l'utilisateur : combien d'étapes supplémentaires le mécanisme impose-t-il ? Existe-t-il un chemin de contournement plus simple mais moins sûr que l'utilisateur empruntera naturellement sous pression de temps ? Les concepteurs expérimentés cherchent systématiquement à faire correspondre le chemin le plus court pour l'utilisateur avec le chemin le plus sûr, plutôt que de multiplier les frictions en espérant que la discipline individuelle compense un mauvais choix de conception.

7. Articulation avec le reste du programme

Ce module synthétise et opérationnalise des notions vues ailleurs dans le programme : vulnérabilités identifiées en *Audit organisation et technique*, gestion des risques du module *Gouvernance*, primitives cryptographiques à intégrer correctement (*Cryptographie*). *Security by design* est la discipline qui transforme ces connaissances en pratiques de conception concrètes, en amont du développement.

Le tableau suivant résume comment chaque chapitre de ce module s'articule avec les autres briques du programme :

Chapitre de ce module	Lien avec le reste du programme
2. Modélisation des menaces	Utilise les catégories de vulnérabilités et de scénarios d'attaque étudiées en <i>Audit organisation et technique</i> comme matière première pour l'identification systématique des menaces
3. Principes de conception sécurisée	S'applique directement aux recommandations correctives issues d'un audit technique
4. Architecture zero trust	Prolonge la segmentation réseau déjà abordée en <i>Audit organisation et technique</i>
5. DevSecOps	Opérationnalise la gestion des vulnérabilités et des correctifs dans un cycle de développement outillé
6. Privacy by design	S'articule avec les obligations réglementaires et la gestion des risques du module <i>Gouvernance</i>

À retenir

- *Security by design* intègre la sécurité dès la conception, réduisant fortement le coût de correction par rapport à une approche corrective a posteriori, car une faille de conception corrigée tôt évite l'effet d'engagement architectural, de régression et d'urgence post-incident propres aux corrections tardives.
- Le modèle de sécurité périmétrique seul est insuffisant face à des attaquants capables de franchir la frontière ; une conception résiliente à la compromission partielle est nécessaire, à l'image d'une résistance structurelle répartie plutôt que concentrée sur un unique point de contrôle.
- Une mesure de sécurité qui dégrade excessivement l'usage est souvent contournée : la sécurité par défaut et simple d'usage est un objectif de conception, pas un vœu pieux — le chemin le plus simple pour l'utilisateur doit coïncider avec le chemin le plus sûr.
- *Security by design* et *privacy by design* sont complémentaires mais distincts : le premier protège le système contre des actions malveillantes, le second protège spécifiquement les personnes dont les données sont traitées.
- Ce module transforme des connaissances déjà rencontrées ailleurs dans le programme (audit, gouvernance des risques, cryptographie) en pratiques concrètes de conception, applicables en amont du développement.

Chapitre 2 — Modélisation des menaces (threat modeling)

1. Qu'est-ce que le threat modeling ?

Le threat modeling est une démarche structurée visant à identifier, de façon systématique et le plus en amont possible, les menaces plausibles pesant sur un système, afin de concevoir des mesures de sécurité proportionnées avant l'implémentation. C'est l'un des outils centraux de *security by design*.

Il s'agit d'un exercice d'anticipation plutôt que de détection : contrairement à un test d'intrusion, qui évalue un système déjà existant en cherchant à l'exploiter concrètement, le threat modeling s'exerce idéalement sur une représentation encore abstraite du système (schéma d'architecture, diagramme de flux) avant même que le code ne soit écrit. Cette différence de nature explique pourquoi threat modeling et tests d'intrusion sont complémentaires plutôt que substituables : le premier balaie systématiquement l'espace des menaces plausibles à moindre coût, le second valide empiriquement la résistance effective d'un système réel, mais seulement sur les chemins que le testeur choisit d'explorer dans le temps imparti.

2. La démarche générale en quatre questions (Adam Shostack)

1. **Sur quoi travaillons-nous ?** (modéliser le système : diagramme de flux de données, composants, frontières de confiance).
2. **Qu'est-ce qui peut mal tourner ?** (identification systématique des menaces).
3. **Que faisons-nous à ce sujet ?** (définir des contre-mesures).
4. **Avons-nous fait du bon travail ?** (valider la démarche, itérer).

Cette formulation en quatre questions, popularisée par Adam Shostack (auteur de référence sur le sujet, ancien responsable du programme de threat modeling chez Microsoft), a l'avantage de rester indépendante de tout outillage ou méthodologie spécifique : elle peut se mener sur un tableau blanc avec des post-it, comme dans un atelier outillé avec un logiciel dédié de modélisation. La quatrième question est souvent la plus négligée en pratique : un threat modeling mené une seule fois, au lancement du projet, puis jamais révisé, perd rapidement sa pertinence à mesure que l'architecture évolue.

3. Diagramme de flux de données (DFD) et frontières de confiance

Avant d'identifier des menaces, il faut représenter le système : processus, flux de données, magasins de données, entités externes, et surtout les **frontières de confiance** (points où le niveau de confiance change, ex. entre un client non authentifié et un serveur, entre une DMZ et un réseau interne). Les menaces se concentrent typiquement sur ces frontières.

Un diagramme de flux de données utilise conventionnellement quatre types d'éléments :

Élément	Symbole usuel	Exemple
Entité externe	rectangle	navigateur d'un utilisateur, système tiers partenaire
Processus	cercle ou ellipse	serveur d'application, service d'authentification
Magasin de données	deux traits parallèles	base de données, système de fichiers, cache
Flux de données	flèche	requête HTTP, appel API interne, écriture en base

Les frontières de confiance sont représentées par une ligne en pointillés traversant le diagramme, séparant des zones de niveaux de confiance différents (ex. Internet public / DMZ / réseau interne / zone d'administration). En pratique, l'expérience montre que la majorité des menaces exploitables se situent précisément au croisement d'un flux de données et d'une frontière de confiance — c'est-à-dire aux points où une donnée ou une requête passe d'un contexte de confiance à un autre, et où une vérification (authentification, autorisation, validation d'entrée) doit impérativement avoir lieu.

4. STRIDE : classification des menaces

Modèle développé par Microsoft, six catégories de menaces à examiner systématiquement pour chaque composant/flux du diagramme :

Lettre	Menace	Propriété de sécurité visée	Exemple
S	Spoofing (usurpation d'identité)	Authenticité	se faire passer pour un autre utilisateur
T	Tampering (falsification)	Intégrité	modifier des données en transit ou au repos
R	Repudiation (répudiation)	Non-répudiation	nier avoir effectué une action, en l'absence de traçabilité
I	Information disclosure (divulgaration d'information)	Confidentialité	accéder à des données sans autorisation
D	Denial of Service (dénier de service)	Disponibilité	rendre un service indisponible
E	Elevation of Privilege (élévation de privilège)	Autorisation	obtenir des droits supérieurs à ceux accordés

Pour chaque flux/composant du diagramme, on se demande systématiquement : ce composant est-il vulnérable à S ? à T ? etc.

Exemple d'application de STRIDE

Considérons, à titre pédagogique, un flux simple : un formulaire de connexion d'une application fictive « GestionCours » envoyant les identifiants saisis par l'utilisateur vers un service d'authentification, qui interroge ensuite une base de données de comptes. L'application systématique de STRIDE à ce flux unique peut donner :

Catégorie STRIDE	Menace identifiée sur ce flux	Contre-mesure envisageable
Spoofing	un attaquant intercepte des identifiants valides et se connecte à la place de l'utilisateur légitime	authentification à double facteur, protection contre les attaques par force brute
Tampering	les identifiants sont modifiés en transit entre le navigateur et le serveur	chiffrement du canal (TLS)
Repudiation	un utilisateur nie s'être connecté à une date donnée	journalisation horodatée et intégrée des tentatives de connexion
Information disclosure	un message d'erreur de connexion révèle si un identifiant existe (« mot de passe incorrect » vs « compte inexistant »)	messages d'erreur génériques ne distinguant pas les deux cas
Denial of Service	un attaquant sature le service d'authentification par un grand nombre de tentatives	limitation de débit (<i>rate limiting</i>), verrouillage temporaire progressif
Elevation of Privilege	une faille dans le service d'authentification permet d'obtenir directement un jeton avec des droits d'administrateur	validation stricte côté serveur des droits attribués, indépendamment de toute donnée fournie par le client

Cet exemple illustre un point méthodologique important : STRIDE ne remplace pas l'expertise du concepteur, il structure et systématise son raisonnement pour éviter d'oublier une catégorie entière de menaces sur un composant donné — l'erreur la plus fréquente en l'absence de méthode étant de se concentrer intuitivement sur une ou deux catégories familières (souvent la divulgation d'information) en négligeant les autres (répudiation, déni de service, en particulier).

5. DREAD : évaluation de la sévérité

Modèle complémentaire (moins utilisé aujourd'hui de façon isolée, souvent remplacé par une matrice de risque classique ou CVSS) pour scorer une menace identifiée selon cinq critères : **D**amage (dommage potentiel), **R**eproducibility (reproductibilité), **E**xploitability (facilité d'exploitation), **A**ffected users (utilisateurs affectés), **D**iscoverability (facilité de découverte).

Chaque critère est généralement noté sur une petite échelle (par exemple de 1 à 3, ou de 1 à 10 selon les variantes), puis les scores sont combinés (souvent par une moyenne) pour obtenir un score global permettant de prioriser les menaces identifiées les unes par rapport aux autres. DREAD a été critiqué pour la subjectivité de la notation (deux évaluateurs peuvent attribuer des scores très différents à la même menace) et pour l'absence de méthodologie publique consensuelle sur la pondération des critères, ce qui explique que de nombreuses organisations lui préfèrent aujourd'hui des référentiels de

scoring plus standardisés, tel le **CVSS** (Common Vulnerability Scoring System) pour les vulnérabilités déjà caractérisées, ou une simple matrice de risque probabilité × impact (vue en *Gouvernance et gestion des risques SI*) adaptée au contexte du threat modeling.

6. Arbres d'attaque (attack trees)

Représentation hiérarchique où le nœud racine est l'objectif de l'attaquant (ex. « compromettre le compte administrateur »), décomposé en sous-objectifs (nœuds enfants) reliés par des relations logiques ET/OU, jusqu'à des actions concrètes réalisables. Cette structure aide à visualiser les différents chemins d'attaque possibles et à identifier où une contre-mesure unique bloque plusieurs branches.

Exemple simplifié d'arbre d'attaque

```
Objectif : Compromettre le compte administrateur (racine)
├── (OU) Deviner ou obtenir le mot de passe
│   ├── (OU) Attaque par force brute sur le formulaire de connexion
│   ├── (OU) Réutilisation d'un mot de passe divulgué lors d'une fuite tierce
│   └── (OU) Hameçonnage ciblé de l'administrateur
├── (OU) Exploiter une vulnérabilité applicative pour contourner l'authentification
│   ├── (ET) Trouver un point d'injection ET obtenir une élévation de privilège
│   └── (OU) Exploiter une session administrateur non expirée sur un poste partagé
└── (OU) Compromettre un poste de travail de l'administrateur
    ├── (OU) Logiciel malveillant via une pièce jointe
    └── (OU) Vulnérabilité non corrigée du système d'exploitation du poste
```

La lecture d'un tel arbre permet d'identifier des contre-mesures à fort effet de levier : dans l'exemple ci-dessus, l'authentification à double facteur affaiblit simultanément plusieurs branches liées à l'obtention du mot de passe (force brute, réutilisation, hameçonnage), sans nécessiter de traiter chaque branche isolément par une mesure dédiée. C'est un des intérêts pédagogiques majeurs des arbres d'attaque : ils rendent visibles les contre-mesures qui protègent contre plusieurs scénarios à la fois, information moins immédiate dans une simple liste de menaces STRIDE traitées composant par composant.

7. PASTA et autres méthodologies

D'autres méthodologies existent, orientées différemment : **PASTA** (Process for Attack Simulation and Threat Analysis) adopte une perspective centrée sur le risque métier et simule des scénarios d'attaque complets ; **LINDDUN** est une méthodologie spécifiquement dédiée aux menaces sur la vie privée (à mettre en lien avec le chapitre 6, *privacy by design*).

PASTA se distingue de STRIDE par son point de départ : alors que STRIDE part du système (diagramme technique) pour remonter vers les menaces, PASTA part explicitement des objectifs métier et des impacts business, en sept étapes (définition des objectifs, définition du périmètre technique, décomposition de l'application, analyse des menaces, identification des vulnérabilités, modélisation des attaques, analyse des risques et de l'impact résiduel). Cette orientation en fait une méthodologie plus lourde mais souvent mieux adaptée à des discussions avec des parties prenantes non techniques

(direction, propriétaires métier d'une application), pour qui un scénario d'attaque exprimé en termes d'impact financier ou réputationnel est plus parlant qu'une liste technique de catégories STRIDE.

8. Quand et comment mener un threat modeling

- Idéalement en phase de conception, avant l'écriture du code, et **révisé** à chaque évolution architecturale significative.
- En atelier collaboratif, associant développeurs, architectes et personnel sécurité — le threat modeling gagne à ne pas être l'exercice isolé d'un seul expert sécurité déconnecté de l'équipe de développement.
- Documenté et priorisé, pour alimenter directement le backlog de développement (contre-mesures = user stories/tâches techniques).

Intégration au cycle de vie du projet

Dans une organisation mature, le threat modeling n'est pas un événement isolé mais un point de contrôle récurrent, déclenché par des événements précis plutôt que par un calendrier arbitraire : ajout d'une nouvelle fonctionnalité traitant des données sensibles, modification significative de l'architecture (ajout d'un nouveau composant externe, changement de fournisseur Cloud), ou constat d'un incident de sécurité nécessitant une réévaluation. Cette approche évite deux écueils symétriques : le threat modeling réalisé une seule fois au lancement puis jamais mis à jour (qui devient rapidement obsolète), et le threat modeling refait intégralement à chaque itération mineure (qui consomme un temps disproportionné par rapport au bénéfice).

À retenir

- Le threat modeling identifie systématiquement les menaces à partir d'une représentation du système et de ses frontières de confiance, avant l'implémentation ; il est complémentaire, et non substituable, aux tests d'intrusion menés a posteriori.
- La démarche en quatre questions d'Adam Shostack (sur quoi travaillons-nous, qu'est-ce qui peut mal tourner, que faisons-nous, avons-nous bien fait) structure l'exercice indépendamment de tout outillage spécifique.
- STRIDE structure l'identification des menaces par catégorie, en miroir des propriétés de sécurité visées ; son application systématique, composant par composant et flux par flux, évite l'oubli intuitif de catégories entières de menaces.
- Les arbres d'attaque rendent visibles les contre-mesures à fort effet de levier, capables de neutraliser plusieurs chemins d'attaque simultanément.
- Les méthodologies alternatives (PASTA, LINDDUN) offrent des perspectives complémentaires selon le contexte : PASTA pour ancrer l'analyse dans le risque métier, LINDDUN pour les risques spécifiques à la vie privée.
- Le threat modeling doit être révisé à chaque évolution architecturale significative, pas figé au lancement du projet.

Chapitre 3 — Principes de conception sécurisée

0. Origine : les travaux de Saltzer et Schroeder

La majorité des principes présentés dans ce chapitre trouvent leur origine dans l'article fondateur de Jerome Saltzer et Michael Schroeder, *The Protection of Information in Computer Systems* (1975), qui a formalisé pour la première fois un ensemble cohérent de principes de conception applicables à tout système informatique cherchant à protéger de l'information. Ce qui frappe, près de cinquante ans plus tard, est la robustesse de ces principes face à l'évolution radicale des technologies : conçus à l'ère des systèmes à temps partagé centralisés, ils s'appliquent sans perte de pertinence aux architectures Cloud distribuées, aux microservices et aux API modernes. Cela s'explique par leur nature : ce ne sont pas des recommandations techniques liées à une technologie donnée, mais des propriétés structurelles générales que tout système de contrôle d'accès doit respecter pour rester robuste.

1. Moindre privilège (least privilege)

Chaque composant, utilisateur ou processus ne doit disposer que des droits strictement nécessaires à l'accomplissement de sa tâche, ni plus. Application concrète : un service applicatif se connectant à une base de données ne doit pas utiliser un compte disposant de droits d'administration complets si seules des opérations de lecture/écriture sur des tables spécifiques sont nécessaires. Ce principe limite l'impact d'une compromission (un composant compromis ne peut nuire qu'à hauteur de ses propres privilèges).

Illustration : compte de service en base de données

```
-- À éviter : le service applicatif utilise un compte disposant de tous les droits
-- GRANT ALL PRIVILEGES ON *.* TO 'service_app'@'%';

-- Application du moindre privilège : droits limités aux tables et opérations nécessair
CREATE USER 'service_app'@'10.0.1.%' IDENTIFIED BY '****';
GRANT SELECT, INSERT, UPDATE ON gestioncours.inscriptions TO 'service_app'@'10.0.1.%';
GRANT SELECT ON gestioncours.etudiants TO 'service_app'@'10.0.1.%';
-- Aucun droit DELETE, DROP, ni accès aux autres bases du serveur
```

Ce même principe s'applique au-delà des bases de données : rôles IAM Cloud limités à un périmètre précis de ressources, jetons d'API à portée (scope) restreinte, comptes de service dédiés à une seule fonction plutôt qu'un compte générique partagé par plusieurs composants. Un corollaire souvent négligé du moindre privilège est sa dimension **temporelle** : un privilège élevé accordé de façon permanente pour un besoin ponctuel (par exemple, un accès administrateur donné à un développeur pour une opération de maintenance unique) devrait être révoqué une fois le besoin satisfait, plutôt que laissé actif indéfiniment « au cas où ». C'est l'idée d'accès *juste-à-temps* (just-in-time), reprise et approfondie au chapitre 4 dans le contexte du zero trust.

2. Défense en profondeur (defense in depth)

Superposer plusieurs couches de contrôles indépendantes, de sorte que la défaillance d'une seule couche ne compromette pas l'ensemble du système. Exemple pour une application web : pare-feu réseau, pare-feu applicatif (WAF), validation des entrées côté serveur, requêtes préparées contre l'injection SQL, chiffrement des données sensibles au repos, journalisation et détection d'anomalies. Aucune couche seule n'est infaillible ; leur combinaison réduit fortement la probabilité qu'une seule faille suffise à compromettre le système entier.

Une condition essentielle pour que la défense en profondeur soit réellement efficace est l'**indépendance** des couches : si toutes les couches de protection reposent sur la même hypothèse ou la même technologie sous-jacente, une faille unique dans cette hypothèse commune peut les neutraliser simultanément. Par exemple, si le pare-feu réseau, le pare-feu applicatif et la validation des entrées reposent tous implicitement sur la même liste blanche de caractères autorisés mal conçue, une technique de contournement de cette liste peut traverser les trois couches d'un coup. Une défense en profondeur robuste combine des mécanismes de nature différente (réseau, applicatif, cryptographique, organisationnel/procédural) plutôt que la simple répétition du même type de contrôle.

3. Échec sécurisé (fail-safe / fail-secure)

En cas d'erreur ou de défaillance, un système doit basculer vers l'état le plus sûr, pas vers un état permissif. Exemple : si un mécanisme de vérification d'autorisation échoue techniquement (exception, timeout), le comportement par défaut doit être de **refuser** l'accès, jamais de l'accorder par défaut. Une erreur de logique inversant ce principe (`if erreur: autoriser_acces()` au lieu de refuser) est une classe de bug de sécurité récurrente.

```
# Anti-pattern à éviter : en cas d'erreur, l'accès est accordé par défaut
def verifier_acces(utilisateur, ressource):
    try:
        return service_autorisation.est_autorise(utilisateur, ressource)
    except ErreurService:
        return True # dangereux : une panne du service d'autorisation ouvre l'accès

# Application du principe d'échec sécurisé : en cas d'erreur, l'accès est refusé
def verifier_acces(utilisateur, ressource):
    try:
        return service_autorisation.est_autorise(utilisateur, ressource)
    except ErreurService:
        journaliser_incident("service d'autorisation indisponible")
        return False # refus par défaut, même au prix d'une indisponibilité fonctionnelle
```

Ce principe illustre un arbitrage fréquent en conception : privilégier la sécurité (refus par défaut) peut dégrader temporairement la disponibilité fonctionnelle du service en cas de panne d'un composant tiers. C'est un compromis assumé : une indisponibilité temporaire, bien que gênante, est presque toujours préférable à un accès non autorisé accordé par erreur, dont les conséquences peuvent être bien plus difficiles à corriger a posteriori (fuite de données, action malveillante déjà réalisée).

4. Médiation complète (complete mediation)

Chaque accès à une ressource protégée doit être vérifié systématiquement, à chaque tentative, sans mise en cache non maîtrisée d'une autorisation antérieure qui pourrait être devenue caduque (ex. droit révoqué entre-temps). Une vérification effectuée une seule fois « à l'entrée » puis considérée valide pour toute la suite d'une session peut devenir obsolète si les droits changent en cours de session.

Un cas fréquent de violation de ce principe se rencontre dans des applications qui vérifient les droits d'un utilisateur au moment de l'affichage d'une page ou d'un menu (par exemple, en masquant un bouton « supprimer » pour un utilisateur non autorisé), mais omettent de revérifier ces mêmes droits côté serveur au moment où l'action est effectivement exécutée. Un attaquant peut alors contourner la vérification côté affichage (interface utilisateur) et invoquer directement l'action côté serveur (appel API direct), en l'absence de médiation complète effective à ce niveau. La règle de conception qui en découle est simple à énoncer mais souvent négligée en pratique : **toute vérification d'autorisation effectuée uniquement côté client (navigateur) doit être considérée comme un confort d'expérience utilisateur, jamais comme un contrôle de sécurité** — le contrôle de sécurité réel doit systématiquement être répété côté serveur.

5. Séparation des tâches (separation of duties)

Répartir des opérations sensibles entre plusieurs personnes ou composants distincts, de sorte qu'aucune entité unique ne puisse, seule, réaliser une action critique de bout en bout sans contrôle croisé. Exemple organisationnel : la personne qui valide un paiement ne doit pas être la même que celle qui l'initie. Exemple technique : séparer les clés de signature de code des systèmes de build automatisés accessibles à de nombreux développeurs.

Dans un pipeline de développement logiciel moderne, ce principe se traduit par exemple par la séparation entre les droits de proposer une modification de code (ouverture d'une *pull request*) et les droits de l'approuver et de la fusionner dans la branche principale (revue de code obligatoire par une personne distincte de l'auteur), ou par la séparation entre les droits de développement d'une application et les droits d'accès direct à l'environnement de production. Cette séparation ne vise pas seulement à se prémunir contre des actions délibérément malveillantes d'un employé (scénario rare), mais surtout à réduire le risque d'erreur humaine non intentionnelle : un second regard indépendant détecte plus souvent une erreur de configuration ou de logique qu'un contrôle purement automatisé.

6. Simplicité de conception (economy of mechanism)

Un système simple est plus facile à analyser, auditer et prouver correct qu'un système complexe. La complexité est l'ennemie de la sécurité : elle multiplie les cas limites, les interactions imprévues entre composants et la surface d'attaque effective. Ce principe justifie de préférer des architectures et des dépendances minimales à des solutions sur-conçues « au cas où ».

Ce principe entre parfois en tension avec d'autres objectifs de conception légitimes (généricité, extensibilité future, performance) : un mécanisme d'autorisation très générique et configurable, capable de gérer un grand nombre de cas particuliers actuels et futurs, est souvent plus difficile à auditer complètement qu'un mécanisme volontairement plus rigide mais dont chaque cas possible peut être

énuméré et vérifié. Une conception mature ne maximise pas la généricité par principe, mais recherche le niveau de complexité minimal suffisant pour couvrir les besoins réels et raisonnablement anticipés du système, en résistant à la tentation d'ajouter des options ou des cas particuliers « au cas où un besoin futur hypothétique se présenterait ».

7. Conception ouverte (open design) — rappel du principe de Kerckhoffs

Rappel direct du module *Cryptographie* : la sécurité d'un système ne doit pas reposer sur le secret de sa conception, mais sur des mécanismes robustes même connus de l'attaquant (clés, secrets d'authentification), le design lui-même pouvant être examiné publiquement.

Ce principe s'oppose à l'approche dite de « sécurité par l'obscurité » (*security through obscurity*), qui consiste à compter sur le fait qu'un attaquant ignore les détails de fonctionnement interne d'un système pour se protéger (par exemple, un algorithme de chiffrement maison non documenté, ou un format de jeton de session propriétaire supposé difficile à deviner). L'expérience répétée du domaine montre que ce type de protection s'effondre dès qu'un attaquant déterminé investit le temps nécessaire pour analyser le système (rétro-ingénierie), alors qu'un mécanisme conçu pour résister même en connaissance complète de son fonctionnement (algorithmes cryptographiques publics et éprouvés, protocoles standardisés et largement examinés par la communauté) offre une garantie de sécurité bien plus fiable et durable. Cela ne signifie pas qu'il faille publier inutilement des détails d'implémentation sensibles (configuration, adresses internes) : la conception ouverte porte sur les *mécanismes* de sécurité eux-mêmes, pas sur l'ensemble des informations opérationnelles d'un système.

8. Minimisation de la surface d'attaque

Réduire le nombre de points d'entrée exposés (fonctionnalités, ports réseau, API, dépendances tierces) au strict nécessaire. Chaque fonctionnalité, endpoint ou dépendance supplémentaire est une surface d'attaque potentielle additionnelle à sécuriser et maintenir — un principe qui rejoint la minimisation de complexité du point précédent.

En pratique, minimiser la surface d'attaque se traduit par des actions concrètes et souvent simples à mettre en œuvre : désactiver les modules ou services non utilisés d'un serveur, fermer les ports réseau non strictement nécessaires, retirer le code mort ou les fonctionnalités de débogage laissées actives en production (endpoints de diagnostic, interfaces d'administration accessibles sans authentification renforcée), et auditer régulièrement les dépendances tierces effectivement utilisées par rapport à celles déclarées. Une pratique de revue périodique de la surface d'attaque exposée (inventaire des points d'entrée réellement accessibles depuis l'extérieur) permet de détecter la dérive progressive d'un système au fil de ses évolutions, phénomène fréquent lorsque des fonctionnalités temporaires ou expérimentales ne sont jamais retirées après leur usage initial.

9. Valeurs par défaut sécurisées (secure by default)

Un système livré ou installé doit être sécurisé « dès la sortie de la boîte », sans que l'utilisateur ait à activer explicitement les protections de base (ex. chiffrement activé par défaut, comptes par défaut

désactivés, ports non essentiels fermés par défaut). Rappel du chapitre 1 : la sécurité facile et par défaut est plus efficace en pratique que la sécurité techniquement supérieure mais optionnelle.

10. Synthèse : comment ces principes se combinent

Ces principes ne s'appliquent pas de façon isolée : ils interagissent et se renforcent mutuellement dans une conception cohérente. Le tableau suivant illustre quelques interactions typiques entre principes :

Situation de conception	Principes combinés
Un service compromis ne peut accéder qu'à un sous-ensemble limité de données, et toute tentative d'accès hors de ce périmètre est bloquée et journalisée	moindre privilège + médiation complète + défense en profondeur
Une panne du service d'authentification entraîne un refus d'accès généralisé plutôt qu'un accès non contrôlé, limité aux seules fonctions strictement nécessaires du système en mode dégradé	échec sécurisé + minimisation de la surface d'attaque
Une modification de code sensible ne peut être déployée en production qu'après revue par une personne distincte de l'auteur, elle-même sans droit de déploiement direct	séparation des tâches + moindre privilège

À retenir

- Ces principes (Saltzer et Schroeder, et compléments ultérieurs) forment un socle de conception réutilisable indépendamment de la technologie employée, car ils décrivent des propriétés structurelles générales plutôt que des recommandations techniques ponctuelles.
- Ils se combinent : la défense en profondeur suppose l'application cohérente du moindre privilège à chaque couche ; l'échec sécurisé suppose une médiation complète ; la séparation des tâches renforce le moindre privilège à l'échelle organisationnelle.
- La médiation complète impose de ne jamais considérer une vérification côté client comme un contrôle de sécurité suffisant : le contrôle réel doit toujours être répété côté serveur.
- La simplicité de conception (economy of mechanism) entre parfois en tension avec la généricité ou l'extensibilité : une conception mature recherche le niveau de complexité minimal suffisant, pas la généricité maximale.
- La conception ouverte rejette la sécurité par l'obscurité au profit de mécanismes robustes même connus de l'attaquant, tout en distinguant cela de la publication inutile d'informations opérationnelles sensibles.
- Aucun principe pris isolément ne suffit ; c'est leur application systématique et cohérente à l'échelle d'un système qui constitue une véritable démarche *security by design*.

Chapitre 4 — Architecture sécurisée : segmentation et zero trust

1. Le modèle périmétrique traditionnel et ses limites

Le modèle de sécurité « château fort » repose sur un périmètre (pare-feu) séparant un réseau interne considéré de confiance d'un extérieur considéré hostile. Ce modèle, longtemps dominant, s'avère fragile face à des menaces qui n'ont plus besoin de franchir frontalement le périmètre : télétravail massif, usage du Cloud (données et applications hors du périmètre physique traditionnel), compromission initiale par hameçonnage d'un poste interne, prestataires tiers disposant d'accès internes.

Trois évolutions structurelles ont particulièrement affaibli la pertinence du modèle périmétrique au fil du temps :

- **La dissolution du périmètre physique** : lorsque les applications et les données d'une organisation sont hébergées chez un ou plusieurs fournisseurs Cloud, et que les utilisateurs y accèdent depuis des réseaux variés (domicile, déplacement, réseaux publics), la notion même de « frontière du réseau interne » perd une grande partie de son sens opérationnel : il n'existe plus un point unique par lequel canaliser et filtrer l'ensemble du trafic.
- **La sophistication des attaques d'ingénierie sociale** : un attaquant qui parvient à obtenir les identifiants d'un utilisateur légitime, par hameçonnage ciblé, contourne intégralement un pare-feu périmétrique aussi robuste soit-il — le pare-feu ne distingue pas une connexion légitime d'une connexion illégitime effectuée avec des identifiants volés mais valides.
- **La menace interne et les tiers de confiance** : un modèle périmétrique accorde implicitement une confiance élevée à tout ce qui se trouve « à l'intérieur », y compris des prestataires externes disposant d'accès distants, des équipements personnels connectés au réseau interne, ou des employés dont le comportement peut, volontairement ou non (poste compromis à leur insu), devenir une source de risque.

2. Segmentation réseau

Avant même le concept de zero trust, la segmentation réduit l'impact d'une compromission en divisant le réseau en zones distinctes avec un filtrage strict entre elles (rappel du module *Audit organisation et technique*, chapitre 4) : DMZ pour les services exposés, réseau interne cloisonné par fonction ou sensibilité, réseau d'administration isolé du réseau utilisateur. Une segmentation fine limite la **propagation latérale (lateral movement)** d'un attaquant ayant compromis un premier point d'entrée.

La propagation latérale décrit le scénario où un attaquant, après avoir obtenu un accès initial limité (par exemple, un poste utilisateur compromis par hameçonnage), cherche ensuite à étendre progressivement son emprise à d'autres systèmes du réseau interne — serveurs de fichiers, contrôleurs de domaine, bases de données — en exploitant l'absence de cloisonnement entre segments. Une segmentation bien conçue transforme un incident potentiellement catastrophique

(compromission de l'ensemble du système d'information à partir d'un unique poste utilisateur) en un incident circonscrit (compromission limitée au segment initialement atteint), ce qui réduit considérablement le temps et les moyens nécessaires pour contenir et remédier à l'incident.

3. Principes du modèle Zero Trust

Formalisé notamment par le NIST (SP 800-207), le modèle zero trust part du principe qu'**aucune confiance implicite ne doit être accordée du simple fait qu'une requête provient du réseau interne** : chaque accès doit être authentifié, autorisé et vérifié en continu, indépendamment de sa localisation réseau d'origine.

Le document NIST SP 800-207 définit sept principes directeurs pour une architecture zero trust, dont les idées maîtresses peuvent se résumer ainsi : toute ressource de données ou de calcul est considérée comme une ressource à protéger indépendamment de son emplacement réseau ; toute communication doit être sécurisée quelle que soit la localisation du réseau ; l'accès à chaque ressource individuelle est accordé par session, sur la base d'une décision dynamique ; l'accès est déterminé par une politique tenant compte de multiples attributs (identité, application, état de l'appareil) et non uniquement par l'emplacement réseau ; l'organisation surveille et mesure en continu l'intégrité et la posture de sécurité de tous les actifs, propres ou associés ; l'authentification et l'autorisation sont strictes et dynamiques, appliquées avant que tout accès ne soit accordé.

Principes clés

- **Vérification explicite** : authentifier et autoriser systématiquement, en s'appuyant sur toutes les données disponibles (identité, appareil, localisation, comportement).
- **Moindre privilège appliqué dynamiquement** : accès juste-à-temps et juste-suffisant (*just-in-time, just-enough-access*), plutôt que des droits larges accordés une fois pour toutes.
- **Présomption de compromission** (*assume breach*) : concevoir le système comme si un attaquant était déjà présent quelque part dans l'environnement, en minimisant le rayon d'action possible (segmentation fine, micro-segmentation) et en maximisant la détection.

Ces trois principes, mis ensemble, changent fondamentalement la question que pose une architecture zero trust par rapport à un modèle périmétrique. Le modèle périmétrique pose implicitement la question : « cette requête provient-elle de l'intérieur du réseau de confiance ? ». Le modèle zero trust pose systématiquement une question différente, réévaluée à chaque accès : « cette identité précise, sur cet appareil précis, dans ce contexte précis, est-elle autorisée à effectuer cette action précise sur cette ressource précise, maintenant ? ».

4. Composants typiques d'une architecture zero trust

Composant	Rôle
Gestion des identités (IAM)	authentification forte (MFA), gestion centralisée des identités et des droits

Composant	Rôle
Politique d'accès contextuelle	décision d'accès basée sur de multiples signaux (identité, posture de l'appareil, localisation, sensibilité de la ressource), pas uniquement sur l'appartenance réseau
Micro-segmentation	contrôle fin des flux, y compris entre charges de travail internes (pas seulement à la frontière du réseau)
Chiffrement systématique	chiffrement des flux, y compris en interne, sans supposer un réseau interne intrinsèquement sûr
Supervision continue	journalisation et détection comportementale permettant de révoquer un accès en cours de session si un comportement anormal est détecté

Exemple illustratif d'une politique d'accès contextuelle

Le fragment ci-dessous illustre, de façon simplifiée et pédagogique, le type de règle qu'un moteur de politique d'accès zero trust peut évaluer avant d'accorder un accès à une ressource sensible, en combinant plusieurs signaux plutôt qu'une seule condition d'appartenance réseau :

```
# Exemple pédagogique simplifié de politique d'accès contextuelle
politique:
  ressource: "application-notes-etudiants"
  regles_acces:
    - condition:
        identite_authentifiee: true
        facteur_authentification: "MFA"
        appareil_conforme: true          # ex. chiffrement disque actif, correctifs à jour
        role_utilisateur: ["enseignant", "administration"]
      decision: "autoriser"
      duree_session_max: "8h"
    - condition:
        localisation_reseau: "inconnue_ou_a_risque"
        role_utilisateur: ["enseignant", "administration"]
      decision: "autoriser_avec_verification_additionnelle"
      verification_additionnelle: "reauthentification_MFA"
    - condition: "default"
      decision: "refuser"
```

Ce fragment illustre un point essentiel du zero trust : la même identité (un enseignant authentifié) peut se voir accorder un accès direct depuis un contexte jugé habituel, ou au contraire soumise à une vérification additionnelle depuis un contexte jugé plus risqué (nouvel appareil, localisation inhabituelle) — la décision n'est jamais figée une fois pour toutes, elle dépend du contexte évalué à chaque tentative d'accès.

5. Zero trust n'est pas un produit unique

Zero trust est une architecture et une philosophie de conception, pas un produit acheté sur étagère : sa mise en œuvre combine plusieurs briques technologiques existantes (IAM, micro-segmentation,

chiffrement, supervision) orchestrées selon les principes ci-dessus, et s'accompagne nécessairement d'une transformation organisationnelle (revue des processus d'accès, culture de vérification continue).

Cette précision est importante car le terme « zero trust » a fait l'objet d'un usage marketing important de la part de nombreux éditeurs de solutions de sécurité, chacun présentant son propre produit comme une solution zero trust complète. Un achat de produit, à lui seul, ne constitue pas une architecture zero trust : celle-ci résulte d'une démarche de conception cohérente, articulant plusieurs briques technologiques autour de politiques d'accès explicites et d'une gouvernance des identités robuste, bien plus que de l'installation d'un outil particulier.

6. Migration progressive

Une migration complète vers zero trust est rarement un projet « big bang » : elle se conduit généralement de façon incrémentale, en priorisant les actifs les plus critiques (identifiés par la démarche de gestion des risques du module *Gouvernance*), avec des architectures hybrides transitoires combinant éléments périmétriques existants et contrôles zero trust nouveaux.

Une démarche de migration progressive typique suit généralement les grandes étapes suivantes : (1) inventaire des actifs et cartographie des flux existants, condition préalable indispensable puisqu'on ne peut protéger correctement ce que l'on ne connaît pas ; (2) renforcement de la gestion des identités (authentification forte généralisée, centralisation des identités) comme fondation, car la vérification explicite de l'identité est le socle sur lequel reposent tous les autres contrôles zero trust ; (3) déploiement progressif de la micro-segmentation, en commençant par les segments hébergeant les actifs les plus critiques ou les plus exposés ; (4) mise en place de politiques d'accès contextuelles de plus en plus fines, au fur et à mesure que la maturité de supervision augmente ; (5) extension progressive du périmètre couvert, jusqu'à réduire autant que possible la dépendance résiduelle au modèle périmétrique traditionnel. Cette progressivité permet à l'organisation d'apprendre et d'ajuster ses politiques d'accès sans provoquer d'interruption massive de service, tout en démontrant une valeur mesurable dès les premières étapes.

À retenir

- Le modèle périmétrique traditionnel suppose une confiance implicite au réseau interne, une hypothèse de moins en moins tenable (télétravail, Cloud, prestataires tiers, hameçonnage contournant frontalement le pare-feu).
- La segmentation réseau, préalable historique au zero trust, limite déjà fortement la propagation latérale d'un attaquant ayant obtenu un accès initial.
- Le zero trust (formalisé notamment par le NIST SP 800-207) remplace la confiance implicite par une vérification systématique et continue de chaque accès, fondée sur de multiples signaux contextuels et non sur la seule provenance réseau.
- La mise en œuvre combine des briques technologiques existantes (IAM, micro-segmentation, chiffrement, supervision) et se conduit généralement de façon progressive et priorisée, en commençant typiquement par le renforcement de la gestion des identités.
- Zero trust est une démarche de conception et de gouvernance, pas un produit unique acheté sur étagère, malgré un usage marketing fréquent du terme par les éditeurs de solutions.

Chapitre 5 — Cycle de développement sécurisé (DevSecOps)

1. Le cycle de développement sécurisé (Secure SDLC)

Intégrer la sécurité à chaque phase du cycle de vie du développement logiciel, plutôt que de la traiter comme une étape de validation finale isolée :

Phase	Activités de sécurité
Exigences	exigences de sécurité explicites, classification de la sensibilité des données traitées
Conception	threat modeling (chapitre 2), revue d'architecture
Développement	bonnes pratiques de codage sécurisé, revue de code, analyse statique (SAST)
Tests	tests de sécurité automatisés, analyse dynamique (DAST), tests d'intrusion
Déploiement	durcissement de la configuration, gestion sécurisée des secrets
Exploitation	supervision, gestion des correctifs, réponse à incident

Un Secure SDLC bien conçu se distingue d'un cycle de développement classique auquel on aurait simplement ajouté une phase de test de sécurité en fin de projet : dans un Secure SDLC, chaque phase produit des artefacts qui alimentent la suivante (les exigences de sécurité orientent le threat modeling, dont les contre-mesures identifiées deviennent des tâches de développement, dont la couverture est vérifiée par les tests). Cette continuité évite l'écueil fréquent où l'équipe sécurité découvre, au moment des tests finaux, des choix d'architecture profondément ancrés qu'il est devenu trop coûteux de remettre en cause — situation qui ramène directement à la discussion du chapitre 1 sur le coût croissant de la correction tardive.

2. « Shift left » : déplacer la sécurité en amont

Le principe du *shift left* consiste à déplacer les activités de sécurité le plus tôt possible dans le cycle de développement (« vers la gauche » d'une frise chronologique), en cohérence avec l'observation du chapitre 1 : plus une faille est détectée tôt, moins elle coûte à corriger.

Le shift left ne signifie pas seulement « exécuter des outils de scan plus tôt » ; il implique un changement de responsabilité : plutôt que la sécurité soit uniquement l'affaire d'une équipe spécialisée intervenant en fin de cycle, chaque développeur devient acteur de la sécurité du code qu'il produit, au moment même où il le produit — retour d'information immédiat (par exemple, un avertissement de l'éditeur de code ou du pipeline de build dès l'introduction d'un motif de code vulnérable) plutôt que différé de plusieurs semaines ou mois lors d'un audit externe.

3. DevSecOps : intégrer la sécurité dans DevOps

DevOps vise à accélérer et fiabiliser le cycle développement-déploiement par l'automatisation (intégration continue, déploiement continu — CI/CD) et une collaboration étroite entre équipes de développement et d'exploitation. **DevSecOps** étend cette approche en intégrant la sécurité comme responsabilité partagée et automatisée tout au long du pipeline, plutôt que comme une porte de validation manuelle et tardive gérée par une équipe séparée.

Contrôles de sécurité automatisables dans un pipeline CI/CD

Étape du pipeline	Contrôle de sécurité
Commit / pre-commit	détection de secrets codés en dur (clés API, mots de passe) avant qu'ils n'entrent dans l'historique du dépôt
Build	analyse statique du code (SAST)
Dépendances	analyse de composition logicielle (SCA) : vulnérabilités connues dans les bibliothèques tierces utilisées
Tests	tests de sécurité automatisés (cas de test dérivés du threat modeling)
Déploiement en environnement de test	analyse dynamique (DAST) contre l'application en fonctionnement
Image de conteneur	analyse de vulnérabilités de l'image avant publication
Infrastructure as Code	analyse de configuration (détection de règles de pare-feu trop permissives, de stockage mal configuré) avant provisionnement

Exemple pédagogique d'extrait de pipeline CI/CD

L'extrait suivant illustre, de façon simplifiée, comment plusieurs de ces contrôles peuvent être intégrés comme étapes successives d'un pipeline d'intégration continue :

```

# Extrait pédagogique simplifié d'un pipeline CI/CD (syntaxe générique)
stages:
  - detection_secrets
  - analyse_statique
  - analyse_dependances
  - tests
  - analyse_image_conteneur
  - analyse_infrastructure_as_code
  - deploiement

detection_secrets:
  stage: detection_secrets
  script:
    - scanner-secrets --chemin ./src --echec-si-trouve

analyse_statique:
  stage: analyse_statique
  script:
    - outil-sast --chemin ./src --seuil-echec "critique,eleve"

analyse_dependances:
  stage: analyse_dependances
  script:
    - outil-sca --fichier-dependances requirements.txt --seuil-echec "critique"

tests:
  stage: tests
  script:
    - executer-tests-unitaires
    - executer-tests-securite-derivees-threat-modeling

analyse_image_conteneur:
  stage: analyse_image_conteneur
  script:
    - construire-image -t app:${CI_COMMIT_SHA}
    - scanner-image app:${CI_COMMIT_SHA} --seuil-echec "critique"

analyse_infrastructure_as_code:
  stage: analyse_infrastructure_as_code
  script:
    - scanner-iac --chemin ./infrastructure --seuil-echec "critique,eleve"

deploiement:
  stage: deploiement
  script:
    - deployer-environnement-test
  when: on_success # ne se déclenche que si toutes les étapes précédentes ont réussi

```

Ce type de pipeline matérialise concrètement le principe de shift left : une vulnérabilité critique introduite dans une dépendance, un secret oublié dans un fichier de configuration, ou une règle de pare-feu excessivement permissive dans un fichier d'infrastructure as code sont détectés automatiquement avant même que le code n'atteigne un environnement partagé, plutôt que découverts a posteriori par un audit ou, pire, par un incident réel en production.

4. Gestion des secrets

Les identifiants, clés d'API et certificats ne doivent jamais être codés en dur dans le code source ou versionnés dans un dépôt Git (même privé — un dépôt peut devenir public par erreur, ou son historique complet reste accessible à quiconque y a eu accès). Bonnes pratiques : gestionnaires de secrets dédiés (Vault et équivalents), variables d'environnement injectées au déploiement, rotation régulière des secrets, scan automatique des dépôts pour détecter des secrets déjà exposés.

```
# Anti-pattern à éviter : secret codé en dur directement dans le code source
CLE_API_PAIEMENT = "<CLE_SECRETE_EN_CLAIR_A_NE_JAMAIS_VERSIONNER>"

# Bonne pratique : le secret est récupéré depuis un gestionnaire de secrets externe,
# jamais présent en clair dans le code source ni dans l'historique du dépôt
CLE_API_PAIEMENT = gestionnaire_secrets.recuperer("cle_api_paiement")
```

Un point souvent sous-estimé par les équipes de développement est la persistance d'un secret compromis dans l'**historique** d'un dépôt Git : même si un secret codé en dur est supprimé dans un commit ultérieur, il demeure récupérable par quiconque a accès à l'historique complet du dépôt (`git log`, clone antérieur, fork existant), tant qu'une réécriture explicite de l'historique n'a pas été effectuée — opération elle-même risquée sur un dépôt partagé. La bonne pratique en cas de secret exposé n'est donc jamais de se contenter de le supprimer dans un nouveau commit, mais de le considérer comme définitivement compromis et de procéder immédiatement à sa **révocation et son remplacement** (rotation), indépendamment du nettoyage de l'historique.

5. Gestion des vulnérabilités des dépendances tierces

La majorité du code d'une application moderne provient de dépendances tierces (bibliothèques open source). Une vulnérabilité découverte dans une dépendance largement utilisée peut affecter un très grand nombre d'applications simultanément (illustré par des incidents majeurs affectant des bibliothèques de journalisation ou de compression largement répandues). L'analyse de composition logicielle (SCA) automatisée en continu, couplée à une politique de mise à jour réactive, est devenue une pratique de sécurité de base incontournable.

Deux dimensions complémentaires structurent une gestion mature des vulnérabilités de dépendances : la **détection** (scan automatique et continu des dépendances directes et transitives — une dépendance transitive étant une dépendance de dépendance, souvent bien plus nombreuse et moins visible que les dépendances directes déclarées par l'équipe de développement elle-même), et la **réactivité** (délai entre la publication d'un correctif pour une vulnérabilité connue et son application effective dans le système). Un inventaire précis et à jour des dépendances utilisées (parfois appelé nomenclature logicielle ou *Software Bill of Materials*, SBOM) facilite considérablement cette seconde dimension : lorsqu'une nouvelle vulnérabilité est publiée pour un composant donné, disposer d'un inventaire exhaustif permet d'identifier immédiatement l'ensemble des systèmes concernés, plutôt que de devoir mener une investigation manuelle dans l'urgence.

6. Culture et organisation

DevSecOps est autant une question de culture que d'outillage : formation des développeurs à la sécurité (« security champions » au sein des équipes de développement), responsabilité partagée plutôt que reportée entièrement sur une équipe sécurité séparée en fin de cycle, et retour d'information rapide (un développeur informé d'une faille immédiatement après son introduction la corrige bien plus efficacement qu'informé plusieurs mois plus tard lors d'un audit externe).

Le modèle des « security champions » mérite d'être détaillé : il s'agit de désigner, au sein de chaque équipe de développement, une ou plusieurs personnes qui, sans être des experts sécurité à temps plein, reçoivent une formation approfondie sur les enjeux de sécurité pertinents pour leur périmètre, et servent de relais de proximité entre l'équipe sécurité centrale et les développeurs au quotidien. Ce modèle répond à un problème d'échelle réel : une petite équipe sécurité centrale ne peut matériellement pas réviser en détail chaque ligne de code produite par l'ensemble des équipes de développement d'une organisation de taille significative ; les security champions démultiplient la capacité de vigilance sécurité sans nécessiter que chaque développeur devienne un expert sécurité généraliste.

Repère : maturité DevSecOps et référentiels d'auto-évaluation

Pour situer la maturité d'une organisation en matière d'intégration de la sécurité dans le développement logiciel, plusieurs référentiels reconnus peuvent être mobilisés, tel **OWASP SAMM** (Software Assurance Maturity Model), qui structure l'évaluation autour de plusieurs fonctions métier (gouvernance, conception, implémentation, vérification, opérations) et de niveaux de maturité croissants pour chacune. L'intérêt pédagogique d'un tel référentiel n'est pas tant d'obtenir un score chiffré que de fournir une grille de lecture structurée permettant à une organisation d'identifier ses pratiques déjà matures et ses angles morts (par exemple, une organisation peut avoir un processus de threat modeling très mature en phase de conception tout en négligeant presque totalement la gestion des vulnérabilités en phase d'exploitation).

À retenir

- Le Secure SDLC intègre des activités de sécurité à chaque phase du développement, pas uniquement en validation finale, avec des artefacts qui se transmettent d'une phase à l'autre (exigences → threat modeling → tâches de développement → tests).
- Le shift left implique un changement de responsabilité, pas seulement un changement de calendrier : chaque développeur devient acteur de la sécurité du code qu'il produit, avec un retour d'information rapide plutôt que différé.
- DevSecOps automatise ces contrôles directement dans le pipeline CI/CD (détection de secrets, SAST, SCA, tests dérivés du threat modeling, DAST, scan d'image de conteneur, scan d'infrastructure as code).
- La gestion des secrets impose la rotation immédiate d'un secret exposé, indépendamment du nettoyage de l'historique du dépôt, qui à lui seul ne suffit pas à neutraliser la compromission.
- La gestion des dépendances tierces vulnérables repose sur deux dimensions complémentaires : la détection continue (y compris des dépendances transitives) et la réactivité de mise à jour, facilitées par un inventaire précis des composants utilisés (SBOM).

- DevSecOps est autant une question de culture que d'outillage : le modèle des security champions démultiplie la capacité de vigilance sécurité sans exiger que chaque développeur devienne expert sécurité généraliste ; des référentiels comme OWASP SAMM aident à situer la maturité d'une organisation.

Chapitre 6 — Privacy by design

1. Définition et origine

Le *privacy by design* (protection de la vie privée dès la conception) est un cadre formalisé par Ann Cavoukian, articulé autour de sept principes fondateurs, visant à intégrer la protection des données personnelles comme caractéristique intrinsèque d'un système, dès sa conception, plutôt que comme un ajout réglementaire tardif.

Ann Cavoukian, alors commissaire à l'information et à la protection de la vie privée de l'Ontario (Canada), a formalisé ce cadre à une époque où la protection de la vie privée était encore très largement traitée comme une question exclusivement juridique et réactive : un formulaire de consentement ajouté après coup, une politique de confidentialité rédigée pour satisfaire une obligation légale plutôt que pour refléter des choix de conception réellement protecteurs. L'apport principal de ce cadre a été de déplacer la question de la vie privée du seul terrain juridique vers le terrain de l'ingénierie et de la conception de systèmes, en affirmant qu'un système peut et doit être conçu, dès son architecture, pour minimiser structurellement les risques pour la vie privée des personnes concernées — une démarche directement parallèle à *security by design*, appliquée spécifiquement à la protection des données personnelles.

2. Les sept principes fondateurs

1. **Proactif, non réactif ; préventif, non correctif** : anticiper les risques pour la vie privée avant qu'un incident ne survienne.
2. **Protection de la vie privée par défaut** : les paramètres les plus protecteurs doivent être activés sans action requise de l'utilisateur.
3. **Protection intégrée à la conception** : composante structurelle du système, pas une fonctionnalité ajoutée après coup.
4. **Fonctionnalité intégrale (« somme positive », pas somme nulle)** : rechercher des solutions où sécurité/fonctionnalité et vie privée coexistent, plutôt que de présenter systématiquement un arbitrage l'un contre l'autre.
5. **Sécurité de bout en bout** : protection tout au long du cycle de vie de la donnée, de la collecte à la suppression.
6. **Visibilité et transparence** : les pratiques réelles doivent être vérifiables, pas seulement déclarées.
7. **Respect de la vie privée des utilisateurs** : conception centrée sur l'intérêt de la personne concernée, avec des interfaces et paramètres clairs et accessibles.

Éclairage sur le principe de « somme positive »

Le quatrième principe mérite un développement particulier car il rectifie une opposition fréquemment présentée comme inévitable entre vie privée et fonctionnalité (ou entre vie privée et sécurité). L'exemple le plus souvent cité pour illustrer ce principe est celui de techniques permettant d'obtenir une

information statistique utile (par exemple, une moyenne agrégée) sans jamais avoir besoin d'accéder aux données individuelles sous une forme directement identifiante (voir la confidentialité différentielle, section 5) : la finalité métier légitime est satisfaite sans sacrifier la protection des personnes concernées. Le principe invite le concepteur à chercher activement ce type de solution « somme positive » avant de conclure qu'un arbitrage strict entre fonctionnalité et vie privée est réellement inévitable dans un cas donné.

3. Traduction juridique : article 25 du RGPD

Le RGPD (Règlement Général sur la Protection des Données, Union européenne, largement pris en référence au-delà de sa portée juridique directe) reprend ce principe dans son article 25 : « protection des données dès la conception et par défaut » (*privacy by design and by default*), imposant concrètement au responsable de traitement de mettre en œuvre des mesures techniques et organisationnelles appropriées dès la conception d'un traitement de données personnelles.

L'article distingue explicitement deux volets complémentaires, qu'il est utile de ne pas confondre : la protection des données **dès la conception** (*by design*), qui impose d'intégrer des mesures de protection dans l'architecture même du système, et la protection des données **par défaut** (*by default*), qui impose que les paramètres appliqués par défaut, en l'absence de toute intervention de l'utilisateur, soient déjà les plus protecteurs (par exemple, un champ de profil optionnel visible uniquement par son propriétaire par défaut, plutôt que public par défaut avec une option de restriction que l'utilisateur devrait découvrir et activer lui-même). Cette distinction rejoint directement le principe de « valeurs par défaut sécurisées » vu au chapitre 3, appliqué spécifiquement au domaine de la protection des données personnelles.

4. Principes de minimisation

- **Minimisation de la collecte** : ne collecter que les données strictement nécessaires à la finalité poursuivie, pas « au cas où » elles seraient utiles plus tard.
- **Minimisation de la conservation** : définir une durée de conservation limitée et justifiée, avec suppression ou anonymisation effective au-delà.
- **Minimisation de l'accès** : appliquer le principe du moindre privilège (chapitre 3) spécifiquement aux données personnelles.
- **Minimisation de l'identifiabilité** : privilégier, quand la finalité le permet, des données pseudonymisées ou anonymisées plutôt que directement identifiantes.

Exemple pédagogique de conception minimisatrice

Considérons, à titre d'illustration, une application fictive de gestion de présence en cours, « PresenceCampus ». Une conception naïve pourrait consister à enregistrer, pour chaque étudiant, l'horodatage précis de chaque entrée et sortie de salle, conservé indéfiniment, accessible à l'ensemble du personnel administratif. Une conception appliquant les principes de minimisation reviendrait plutôt à se poser systématiquement, pour chaque donnée envisagée, la question de sa nécessité réelle par rapport à la finalité déclarée (vérifier la présence à une séance donnée) :

Donnée envisagée	Nécessaire à la finalité « vérifier la présence » ?	Choix de conception minimiseur
Présence/absence par séance (oui/non)	oui, directement	conservée
Horodatage précis à la seconde de l'entrée	non, une granularité par séance suffit	non collecté, ou agrégé au niveau de la séance
Historique complet conservé sans limite de durée	non, une durée limitée à l'année académique suffit généralement	suppression ou anonymisation automatique en fin d'année académique
Accès en lecture par l'ensemble du personnel administratif	non, seul le personnel pédagogique concerné par le cours en a besoin	accès restreint par le principe du moindre privilège

5. Techniques de protection de la vie privée

Technique	Principe
Pseudonymisation	remplacer un identifiant direct par un pseudonyme, la ré-identification restant possible avec une information additionnelle gardée séparément
Anonymisation	rendre la ré-identification impossible, y compris par recoupement avec d'autres sources (plus difficile à garantir réellement que ce que l'on croit souvent)
Chiffrement	protège la confidentialité des données au repos et en transit (lien direct avec le module <i>Cryptographie</i>)
Confidentialité différentielle (differential privacy)	ajout de bruit statistique calibré à des résultats d'agrégation pour empêcher la ré-identification d'un individu tout en préservant l'utilité statistique globale
Contrôle d'accès et journalisation	limiter et tracer qui accède à quelles données personnelles

Pourquoi l'anonymisation est plus difficile qu'il n'y paraît

Il est important d'insister, sur le plan pédagogique, sur la difficulté réelle d'obtenir une anonymisation effective et durable. Un jeu de données dont les identifiants directs (nom, numéro d'identification) ont simplement été retirés n'est pas nécessairement anonyme au sens strict : la combinaison de plusieurs attributs indirects, apparemment anodins pris séparément (par exemple, date de naissance approximative, code postal, profession), peut suffire à ré-identifier une personne de façon quasi unique au sein d'une population, en particulier si ce jeu de données est recoupé avec d'autres sources d'information publiques ou tierces. C'est pourquoi la pseudonymisation (qui conserve explicitement la possibilité théorique de ré-identification via une information additionnelle gardée séparément et protégée) et l'anonymisation véritable (qui doit résister à toute tentative de ré-identification, y compris

par recoupement) sont deux notions à ne pas confondre : la réglementation applicable et les techniques de protection appropriées diffèrent sensiblement selon que l'on se situe dans l'un ou l'autre cas.

6. Analyse d'impact relative à la protection des données (AIPD/DPIA)

Pour un traitement de données présentant un risque élevé pour les droits des personnes, une analyse d'impact formelle est requise par de nombreuses réglementations : elle documente la finalité, la nécessité et la proportionnalité du traitement, évalue les risques pour les personnes concernées, et identifie les mesures pour les atténuer — une démarche directement apparentée à l'analyse de risque du module *Gouvernance et gestion des risques SI*, appliquée spécifiquement à la vie privée.

Une AIPD suit généralement une structure en plusieurs temps qu'il est utile de mettre en parallèle avec la démarche de threat modeling du chapitre 2 : description systématique du traitement envisagé et de ses finalités (comparable à « sur quoi travaillons-nous ? ») ; évaluation de la nécessité et de la proportionnalité du traitement au regard de sa finalité ; identification des risques pour les droits et libertés des personnes concernées (comparable à « qu'est-ce qui peut mal tourner ? ») ; définition des mesures envisagées pour traiter ces risques (comparable à « que faisons-nous à ce sujet ? »). Cette parenté méthodologique n'est pas fortuite : les deux démarches partagent la même logique d'anticipation structurée des risques avant la mise en œuvre effective d'un système ou d'un traitement.

7. LINDDUN : threat modeling orienté vie privée

Complément du chapitre 2 : LINDDUN structure l'identification de menaces spécifiques à la vie privée (Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, Non-compliance), sur le même principe que STRIDE mais orienté vers les atteintes à la vie privée plutôt que vers les propriétés de sécurité classiques.

Lettre	Menace (terme original)	Signification
L	Linkability	possibilité de relier deux éléments d'information ou deux actions à une même personne, sans que cela soit souhaité
I	Identifiability	possibilité d'identifier une personne à partir de données qui devraient rester non identifiantes
N	Non-repudiation	impossibilité pour une personne de nier avoir effectué une action, alors que l'anonymat ou le déni plausible serait légitimement attendu dans ce contexte
D	Detectability	possibilité de déduire l'existence d'une donnée ou d'un événement concernant une personne, même sans en connaître le contenu exact
D	Disclosure of information	divulgaration non autorisée du contenu d'une donnée personnelle
U	Unawareness	la personne concernée n'a pas conscience de la collecte ou de l'usage fait de ses données

Lettre	Menace (terme original)	Signification
N	Non-compliance	le traitement ne respecte pas les obligations légales ou les engagements pris envers la personne concernée

Il est instructif de noter que certaines catégories LINDDUN sont, en un sens, à l'opposé de catégories STRIDE portant un nom voisin : là où STRIDE cherche à garantir la non-répudiation (pouvoir prouver qu'une action a bien été effectuée par une personne donnée, propriété de sécurité recherchée), LINDDUN identifie au contraire la non-répudiation comme une **menace potentielle pour la vie privée** dans certains contextes (par exemple, un système de vote ou de consultation devant garantir l'anonymat de la personne ayant exprimé une opinion). Cette tension illustre bien pourquoi *security by design* et *privacy by design*, bien que complémentaires, ne poursuivent pas systématiquement les mêmes objectifs et peuvent, dans certains cas précis, entrer en tension l'un avec l'autre — un arbitrage de conception à traiter explicitement plutôt qu'à ignorer.

À retenir

- Le *privacy by design* intègre la protection des données personnelles comme caractéristique par défaut d'un système, formalisé juridiquement par l'article 25 du RGPD (qui distingue explicitement la protection « dès la conception » et « par défaut ») et des dispositions équivalentes ailleurs.
- Le principe de « somme positive » invite à chercher activement des solutions où fonctionnalité et vie privée coexistent, plutôt que de présumer un arbitrage strict et inévitable entre les deux.
- La minimisation (collecte, conservation, accès, identifiabilité) est le principe opérationnel le plus directement actionnable en conception, comme l'illustre l'exemple d'une application de gestion de présence appliquant systématiquement le test de nécessité à chaque donnée envisagée.
- L'anonymisation véritable est plus difficile à garantir qu'il n'y paraît, en raison du risque de ré-identification par recoupement d'attributs indirects ; elle se distingue nettement de la pseudonymisation, réversible par nature.
- L'analyse d'impact relative à la protection des données (AIPD/DPIA) partage sa logique méthodologique avec le threat modeling du chapitre 2, appliquée spécifiquement aux risques pour les personnes concernées.
- LINDDUN transpose la logique de threat modeling (STRIDE) spécifiquement aux risques pour la vie privée, révélant au passage que certains objectifs de sécurité (comme la non-répudiation) peuvent, selon le contexte, entrer en tension avec les objectifs de protection de la vie privée plutôt que les renforcer systématiquement.